

Collaborative Git Workflow

Building a Student Information Web Application as a Three-Developer Team

Course	DevOps — MCA, Semester 5
Institution	Christ University, Dept. of Computer Science
Submitted by	Arpan Mukherjee (Team Lead / Developer)
Team size	3 developers
Date	9 September 2026

Repository

<https://github.com/Arpan7125/student-info-app>

1. Learning Objectives

This exercise uses a deliberately simple web application as the vehicle for practising a collaborative Git workflow. The application is not the point; the workflow is. On completion the team can:

- Configure Git locally and verify the configuration
- Create and clone a remote repository on GitHub
- Create, switch between and publish feature branches
- Write meaningful commit messages
- Open, review and merge Pull Requests
- Deliberately create a merge conflict and resolve it
- Work in parallel without overwriting another developer's changes

2. Team and Roles

Student	Role	Responsibility
Arpan Mukherjee 2547116 · @Arpan7125	Team Lead / Developer	Repository creation, base application, reviewing every Pull Request, integration
Team Member 2	UI Developer	HTML structure and CSS styling, contact section, application heading
Team Member 3	JavaScript Developer	Show/Hide details behaviour, application title, conflict resolution

To complete before submission

Replace “Team Member 2” and “Team Member 3” with your teammates' real names, register numbers and GitHub usernames — in this table, in the application's Project Team cards, and in `README.md`.

3. The Application

A single-page Student Information System built from three static files — no database, no backend, no build step. It displays one student record, reveals additional details on demand, and lists the project team.

File	Purpose
<code>index.html</code>	Page structure — student record, details panel, contact links, team cards
<code>style.css</code>	Design tokens, card components, animated backdrop, light and dark themes
<code>script.js</code>	Details expand/collapse and the pointer-tracked card sheen
<code>README.md</code>	Project documentation and workflow record

Technologies used

- **HTML5** — semantic structure (`<d1>` record rows, inline SVG icons)

- **CSS3** — custom-property design tokens, flexbox and grid, keyframe animation, automatic dark mode via `prefers-color-scheme`, `prefers-reduced-motion` support
- **Vanilla JavaScript** — no libraries, no dependencies
- **Git** 2.55.0 and **GitHub** (repository, Pull Requests, code review)
- **GitHub CLI** 2.100.0, **VS Code**, browser

4. Application Screenshots

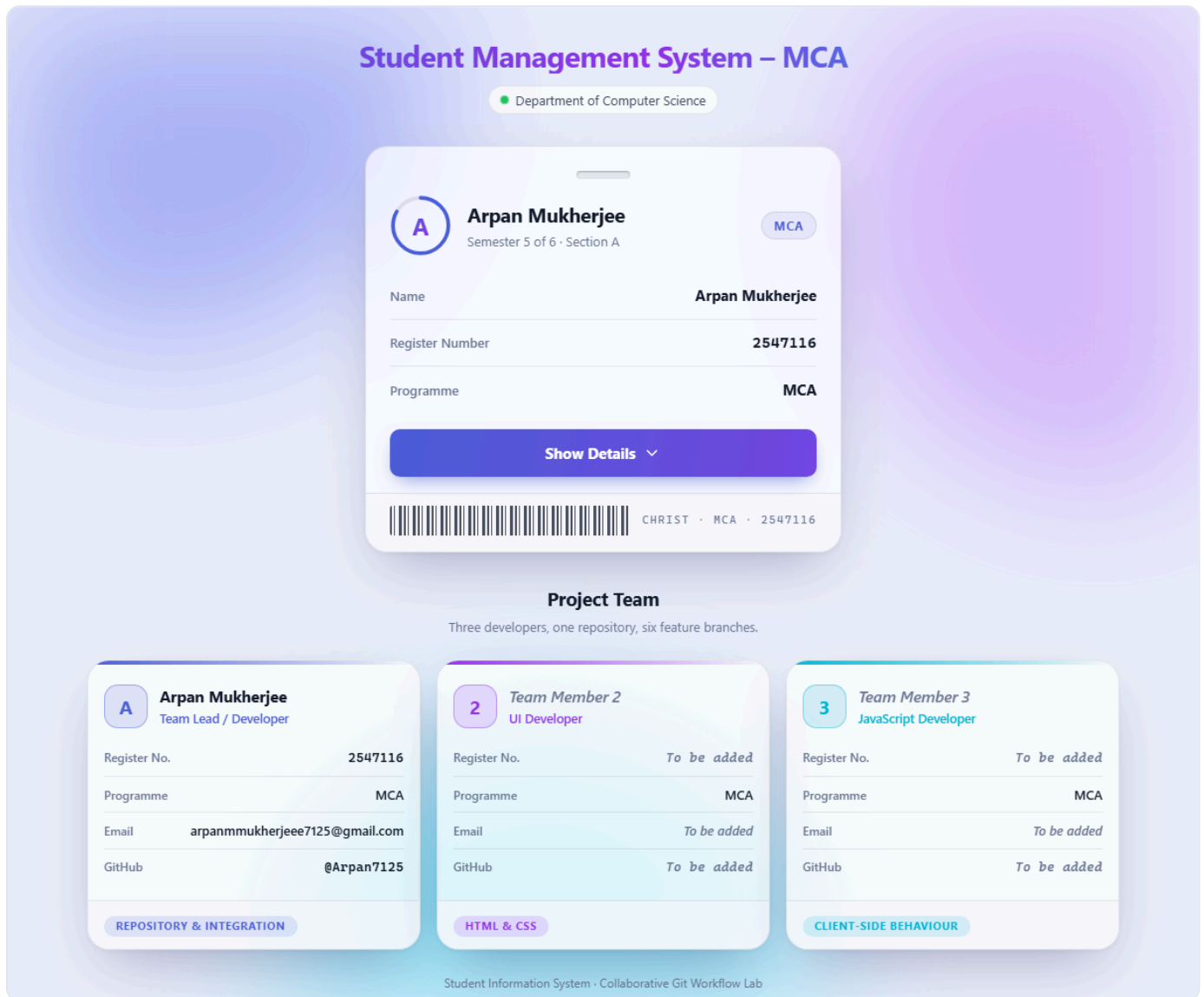


Figure 1 — Default view (light theme). Student record card with semester progress ring and barcode strip, above the three Project Team cards.

Student Management System – MCA

Department of Computer Science

A

Arpan Mukherjee
Semester 5 of 6 · Section A

MCA

Name	Arpan Mukherjee
Register Number	2547116
Programme	MCA

Hide Details ^

Semester	5
Section	A
Mentor	Dr. Cynthia T

CONTACT INFORMATION

Email	arpanmmukherjee7125@gmail.com
Phone	To be added

 CHRIST · MCA · 2547116

Project Team

Three developers, one repository, six feature branches.

A

Arpan Mukherjee
Team Lead / Developer

Register No.	2547116
Programme	MCA
Email	arpanmmukherjee7125@gmail.com
GitHub	@Arpan7125

REPOSITORY & INTEGRATION

2

Team Member 2
UI Developer

Register No.	To be added
Programme	MCA
Email	To be added
GitHub	To be added

HTML & CSS

3

Team Member 3
JavaScript Developer

CLIENT-SIDE BEHAVIOUR

Figure 2 — After clicking **Show Details**: semester, section and mentor slide open, the button becomes **Hide Details** and the chevron rotates.

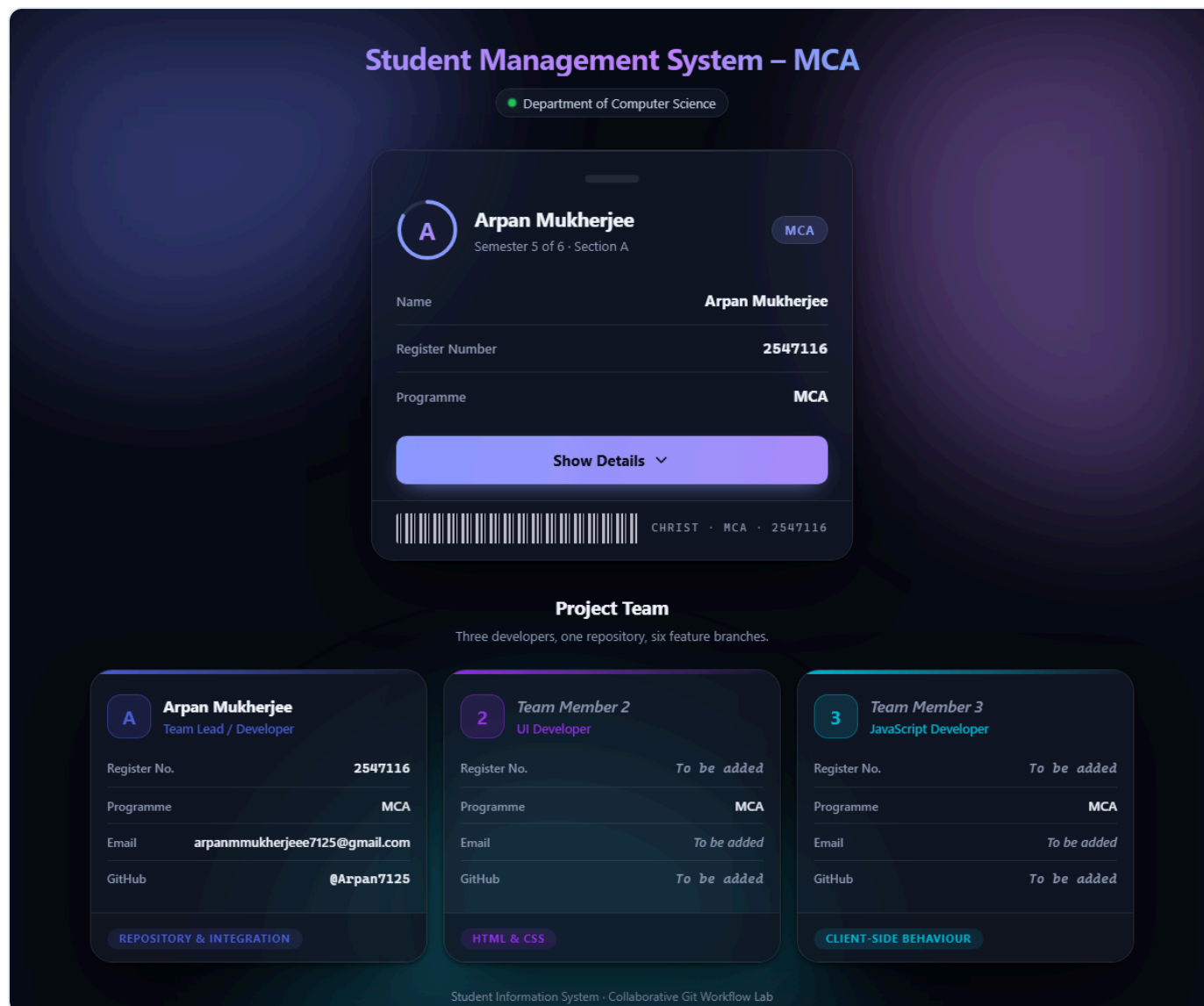


Figure 3 — The same page in dark theme. Every colour is a token redefined under `prefers-color-scheme: dark`; nothing is hardcoded.

Student Management System -

Department of Computer Science



Arpan Mukherjee

Semester 5 of 6 · Section A

Name

Arpan M

Register Number

2

Programme

Show Details ▾



CHRIST · MCA ·

Project Team

Three developers, one repository, six feature branches



Arpan Mukherjee

Team Lead / Developer

Register No.

Programme

Email

arpanmmukherjee7125@

GitHub

@A

REPOSITORY & INTEGRATION



Team Member 2

UI Developer

Register No.

To

Programme

Email

GitHub

To

Student Management System -

Department of Computer Science



Arpan Mukherjee

Semester 5 of 6 · Section A

Name

Arpan M

Register Number

2

Programme

Hide Details ▲

Semester

Section

Mentor

Dr. C

CONTACT INFORMATION



Email

arpanmmukherjee7125@g



Phone

To



CHRIST · MCA ·

Project Team

Three developers, one repository, six feature branches



Arpan Mukherjee

Team Lead / Developer

Register No.

Programme

Email

arpanmmukherjee7125@

GitHub

@A



Figure 4 — Mobile (375 px): cards stack to one column, no horizontal scrolling.

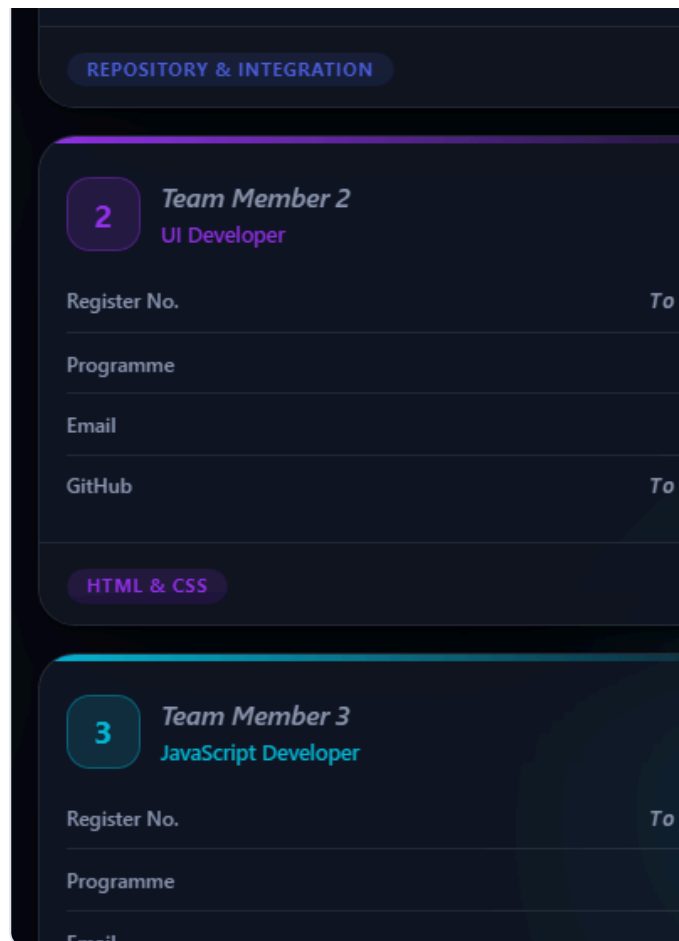


Figure 5 — Mobile, dark theme, details expanded.

5. Git Branching Strategy

`main` is protected by team convention: **nobody commits to it directly**. All work happens on a short-lived feature branch, which is published and merged back through a reviewed Pull Request. Each new branch is created from a freshly pulled `main`.



Branch	Author	Purpose
main	—	Always-working integration branch
feature/ui	Team Member 2	Card layout, spacing, typography, button styling
feature/javascript	Team Member 3	Show/Hide details behaviour
feature/contact	Team Member 2	Contact information section
feature/student-name	Team Member 2	Heading → "Student Management System"
feature/app-title	Team Member 3	Heading → "MCA Student Information Portal"
feature/redesign	Team Member 2	Campus ID-card redesign and Project Team cards

Commands used

```
$ git config --global user.name "Arpan Mukherjee"
$ git config --global user.email "arpanmmukherjee7125@gmail.com"
$ git config --global --list

$ gh repo create student-info-app --public
$ git clone https://github.com/Arpan7125/student-info-app.git
$ cd student-info-app

$ git add .
$ git commit -m "Initial student information app"
$ git push -u origin main

# each developer, for each piece of work:
$ git checkout main
$ git pull origin main
$ git switch -c feature/<name>
# ... edit files ...
$ git add .
$ git commit -m "<meaningful message>"
$ git push -u origin feature/<name>
# then open a Pull Request on GitHub for the team lead to review
```

6. Commit History

Every message states what changed. Messages such as `update`, `changes`, `final` or `latest` were avoided.

```
$ git log --oneline --graph

* 9b8119e Merge pull request #6 from Arpan7125/feature/redesign
|\
| * 914864e Redesign the interface as an animated campus ID card
|/
* b6ea35a Merge pull request #5 from Arpan7125/feature/app-title
|\
| * e65885f Resolve merge conflict in application title
| |\
| |/\
|/|
* | f957fab Merge pull request #4 from Arpan7125/feature/student-name
|\ \
| * | 17cf9f8 Update application heading
|/ /
| * c9c926f Update application title
|/
* bf45ea7 Merge pull request #3 from Arpan7125/feature/contact
|\
| * d72f9ae Add contact information
|/
* 9599709 Merge pull request #2 from Arpan7125/feature/javascript
|\
| * de46f44 Add student details functionality
* | 2dca70b Merge pull request #1 from Arpan7125/feature/ui
|\ \
| |/\
|/|
| * ed57371 Improve student information UI
|/
* 2679d23 Initial student information app
```

The graph reads bottom-up. Each `|\` fork is a feature branch leaving `main`; each merge commit is a Pull Request being accepted.

7. Pull Requests

PR	Branch → main	Author	Status
#1	feature/ui	Team Member 2	MERGED
#2	feature/javascript	Team Member 3	MERGED
#3	feature/contact	Team Member 2	MERGED
#4	feature/student-name	Team Member 2	MERGED
#5	feature/app-title	Team Member 3	MERGED after conflict resolution
#6	feature/redesign	Team Member 2	MERGED

Review checklist applied by the team lead

1. Does the application still work in the browser?
2. Is the code understandable to someone who did not write it?

- 3. Is the change relevant to the branch's stated purpose?
- 4. Is the commit message meaningful?

8. The Merge Conflict

8.1 What caused it

Branches `feature/student-name` and `feature/app-title` were both created from the **same commit** on `main` (`bf45ea7`), and both edited **the same line** of `index.html` :

Branch	Line 11 of index.html
<code>feature/student-name</code>	<code><h1>Student Management System</h1></code>
<code>feature/app-title</code>	<code><h1>MCA Student Information Portal</h1></code>

PR #4 was merged first, so `main` moved forward. When PR #5 was then merged, Git had two different changes to the same line sharing one common ancestor, and no rule for deciding which is correct — so it stopped and asked a human.

```
$ git merge main
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.

$ git status --short
UU index.html
```

8.2 The conflict markers

```
<<<<<< HEAD
    <h1>MCA Student Information Portal</h1>
=====
    <h1>Student Management System</h1>
>>>>>> main
```

Marker	Meaning
<code><<<<<< HEAD</code>	Start of the version on <i>your current branch</i>
<code>=====</code>	Divider between the two versions
<code>>>>>>> main</code>	End of the version coming in <i>from main</i>

8.3 How it was resolved

Rather than discarding either teammate's intention, the two were combined into a single heading. All three marker lines were deleted.

```
$ git checkout main
$ git pull origin main
$ git checkout feature/app-title
$ git merge main          # conflict reported here

# edit index.html - keep the combined heading, remove every marker

$ git add index.html
$ git commit -m "Resolve merge conflict in application title"
$ git push origin feature/app-title
```

Final line in `index.html`:

```
<h1>Student Management System - MCA</h1>
```

The Pull Request then became mergeable and the team lead merged it.

Key takeaway

A merge conflict is not an error and not a failure. It is Git refusing to guess when two developers disagree about the same line — and correctly handing the decision back to the team.

Screenshot to attach here

The conflict was captured live on GitHub while PR #5 was blocked — the page showed *"This branch has conflicts that must be resolved"* with `index.html` listed, and a red

Merge conflicts

badge beside the PR title. Save that image as `docs/screenshots/06-github-conflict.png` and paste it below this paragraph. PR #5 has since been merged, so the banner no longer appears on that page; the merged state and the resolution commit remain visible in the PR history.

9. How to Run the Application

No server, no dependencies and no build step are required.

```
$ git clone https://github.com/Arpan7125/student-info-app.git
$ cd student-info-app
```

Then open `index.html` in any browser — double-click it, or:

```
start index.html    # Windows
open index.html     # macOS
xdg-open index.html # Linux
```

Click **Show Details** to expand the additional student information. The page follows the operating system's light or dark theme automatically.

10. Deliverables Checklist

Deliverable	Required	Status	Notes
Repository with 4 files	4	DONE	index.html, style.css, script.js, README.md
Branches	6	DONE	7 on the remote: main + 6 feature branches
Meaningful commits	≥ 5	DONE	9 commits, all descriptive
Pull Requests	≥ 3	DONE	6 opened, all 6 merged
Conflict evidence	Screenshots	PARTIAL	Terminal output captured; add the GitHub screenshot — section 8.3
README with all sections	Yes	DONE	In the repository
Real teammate names	Yes	PENDING	Placeholders still in place